



CI/CD

PERFORMANCE TESTING PROJECT

DEMOBLAZE APPLICATION
PERFORMANCE TESTING

TOOLS & TECHNOLOGIES USED



GitHub
Version Control



Jenkins
CI/CD Automation



Apache JMeter
Performance Testing



demoblaze
Demoblaze
Application Under Test



HTML Reports
Test Reports & Analytics



PREPARED BY
Srihari Manthena
(20588213)



AUTOMATE



TEST



ANALYZE



OPTIMIZE

Ensuring Performance, Reliability & Continuous Quality

TABLE OF CONTENTS

1. Objective
2. Project Overview
3. Scope of Testing
4. Test Workflow
5. Architecture Overview
6. Correlation
7. Parameterization
8. Test Configuration (Thread Group)
9. Service Level Expectations (SLA)
10. CI/CD Integration
11. Results & Analysis
12. Challenges Faced
13. Conclusion
14. Future Enhancements

CI/CD PERFORMANCE TESTING PROJECT

Demoblaze Performance Testing

Prepared by: Srihari Manthena (20588213)

1. OBJECTIVE:

The objective of this project is to design and implement an automated performance testing framework integrated with CI/CD to:

- Validate application performance under load
- Identify performance bottlenecks
- Ensure system stability during concurrent user activity
- Enable continuous performance testing in the development lifecycle

2. PROJECT OVERVIEW:

This project focuses on simulating real-world user behavior on an e-commerce application using **Apache JMeter**, with execution automated via **Jenkins CI/CD pipeline** and version control managed through **GitHub**.

The test simulates key user actions such as:

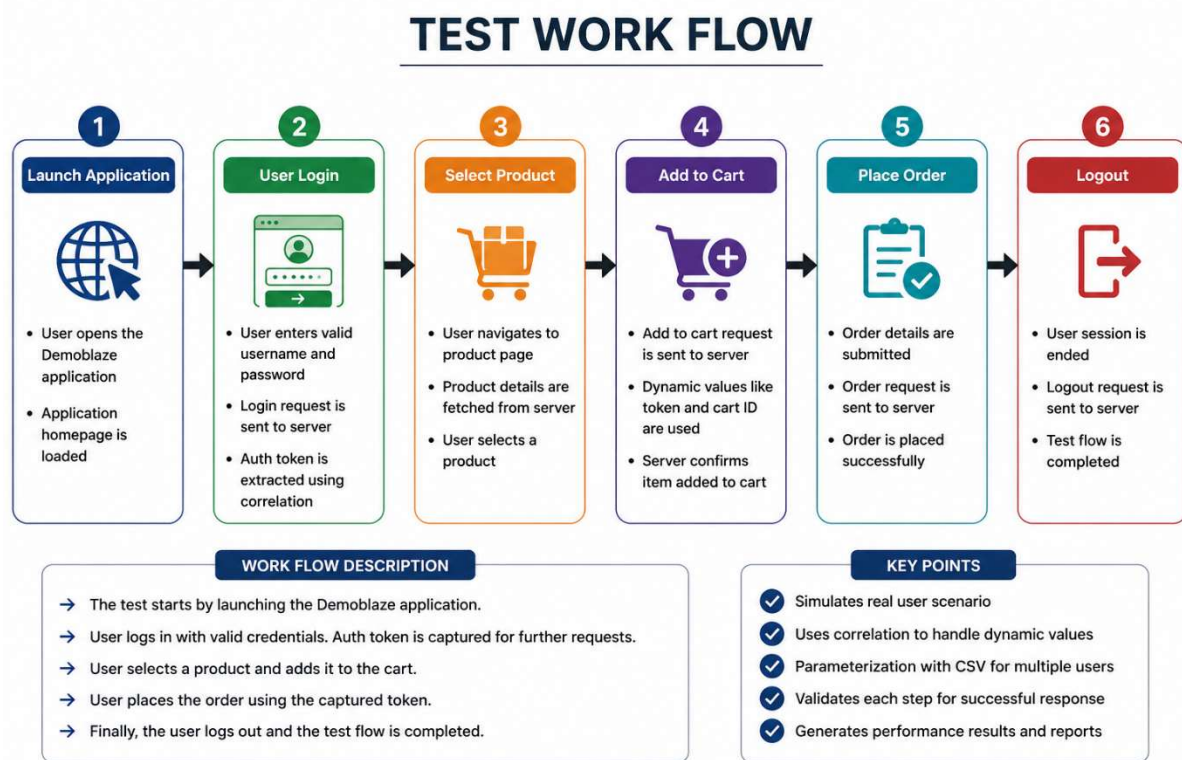
- Application launch
- User login
- Product selection
- Add to cart
- Order placement
- Logout

The framework generates performance reports that help analyze system responsiveness, throughput, and error rates.

3. SCOPE OF TESTING:

- Type of Test: **Load Testing**
- Scenario: End-to-end user journey simulation
- Goal: Evaluate performance under moderate load conditions

4. TEST WORKFLOW:



The test simulates a complete user journey:

1. Launch the application
2. Login using valid credentials
3. Navigate to product page
4. Add product to cart
5. Place order
6. Logout

This ensures realistic performance validation aligned with production user behavior.

5. ARCHITECTURE OVERVIEW:

The project follows a CI/CD-based automated testing architecture:

Flow:

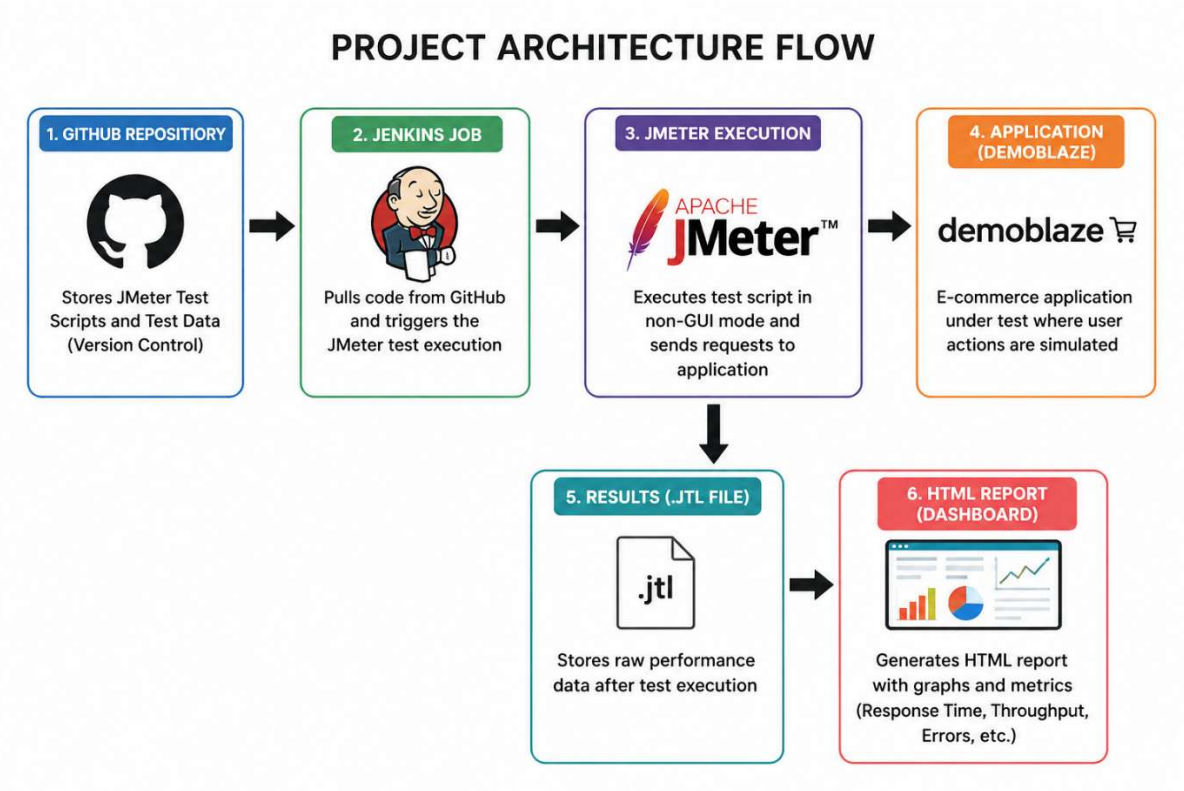
GitHub → Jenkins → JMeter Execution → Results (.jtl) → HTML Report

Components:

- **GitHub**
Stores JMeter scripts and test data (version control)

- **Jenkins**
Automates test execution by pulling latest code from repository
- **Apache JMeter**
Executes performance tests in non-GUI mode
- **Application (Demoblaze)**
Target system under test
- **Reports**
Generates .jtl files and HTML dashboards for analysis

5.1 Architecture Flow Chart



6. CORRELATION

Handled dynamic values like Auth Token, Cookie, and Token using Regex Extractor.

7. PARAMETERIZATION

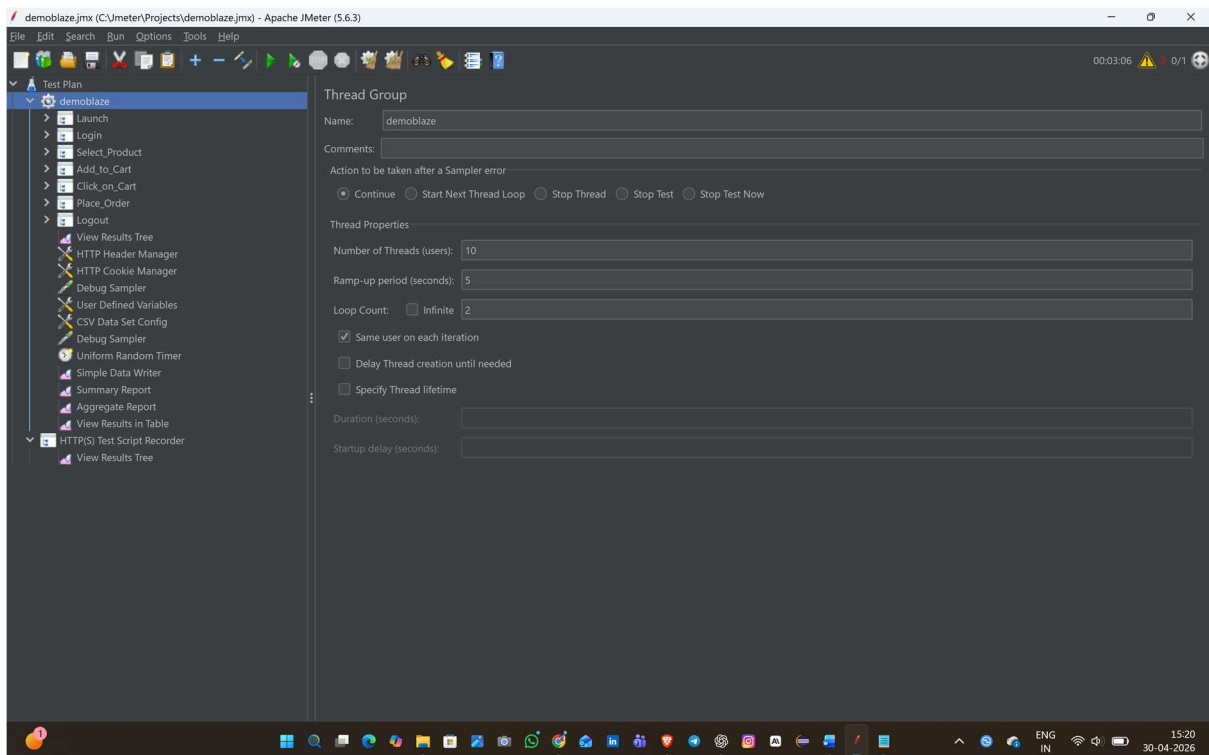
Used CSV Data Set Config to simulate multiple users.

8. TEST CONFIGURATION (THREAD GROUP)

The Thread Group in JMeter defines how many users will simulate the application and how the load is applied.

8.1 Load Testing

Parameter	Value	Description
Number of Threads (Users)	10	Total virtual users simulating the application
Ramp-up Period	5 seconds	Time taken to start all users gradually
Loop Count	2	Number of times each user executes the test
Scheduler	Not Used	Test runs based on loop count, not time duration



9. SERVICE LEVEL EXPECTATIONS (SLA)

Metric	Expected Value
Average Response Time	< 500 ms
Error Rate	< 1%
Throughput Stability	Consistent

10.CI/CD INTEGRATION

10.1 Overview

The project integrates Apache JMeter with Jenkins and GitHub to automate performance testing as part of a CI/CD pipeline. This ensures that tests are executed automatically without manual intervention.

10.2 Integration Flow

GitHub → Jenkins → JMeter Execution → Results → HTML Report

10.3 Implementation Steps

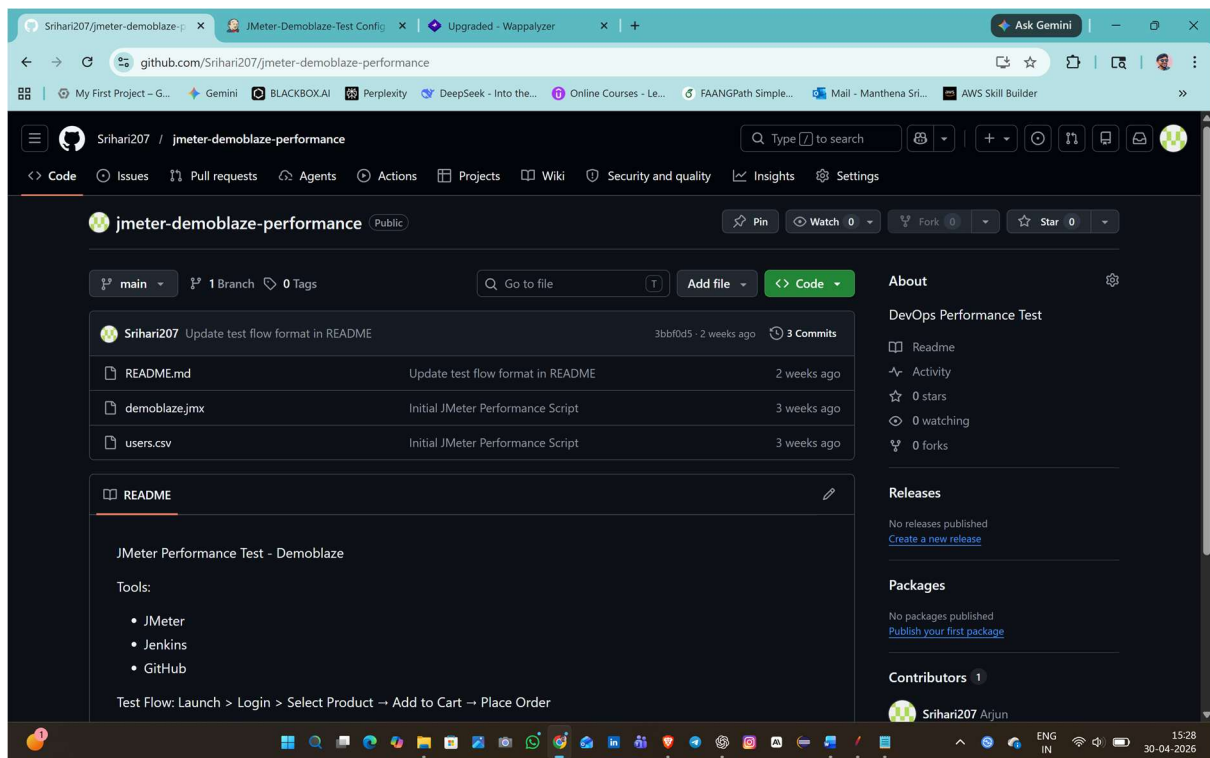
- JMeter test script and CSV data are stored in GitHub repository
 - Jenkins is configured to connect with GitHub
 - Jenkins pulls the latest code from the repository
 - JMeter test is executed in non-GUI mode using command line
 - Results are generated in .jtl format
 - HTML report is created automatically after execution
-

10.4 Jenkins Execution Command

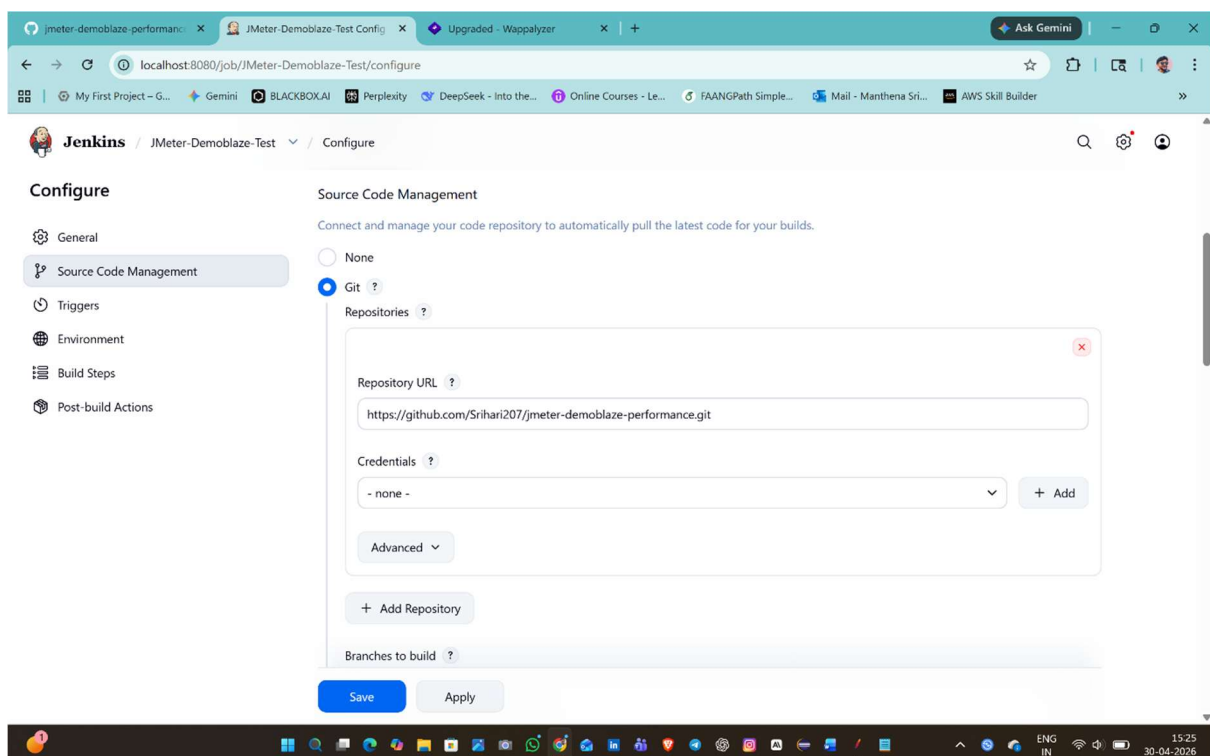
```
jmeter -n -t demoblaze.jmx -l results.jtl -e -o report
```

10.5 Benefits

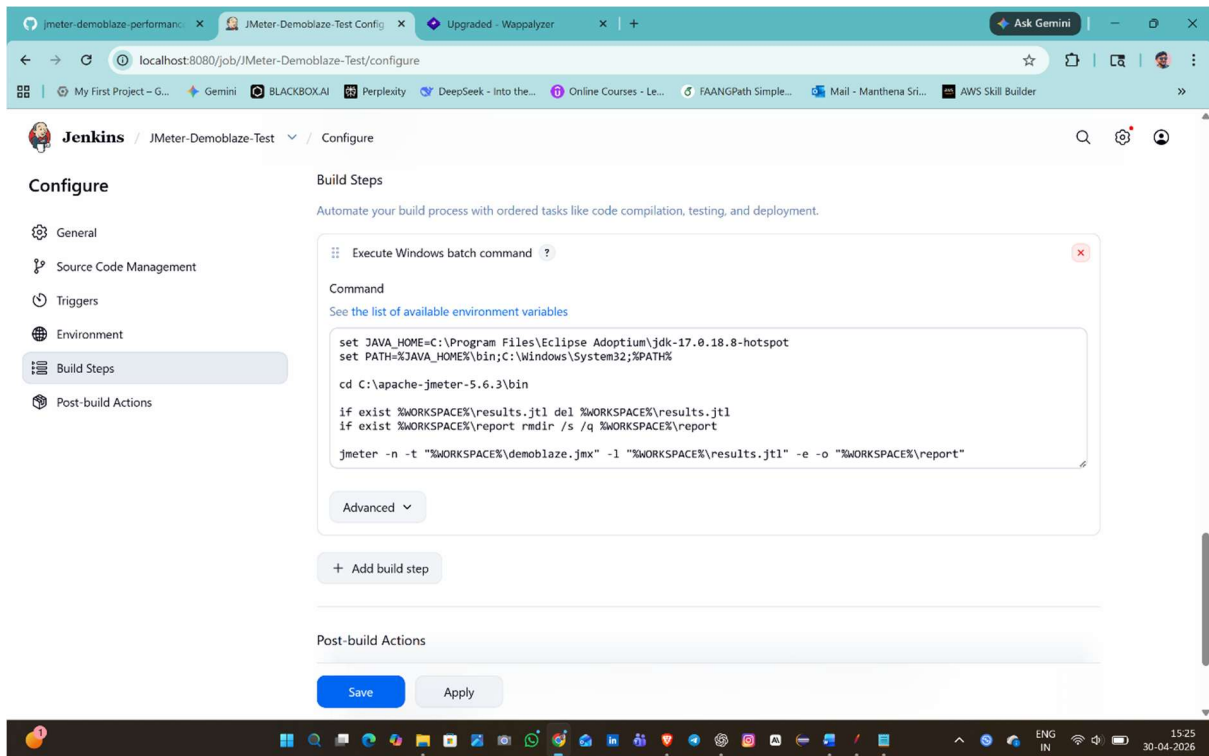
- Automated test execution
- Reduces manual effort
- Enables continuous performance testing



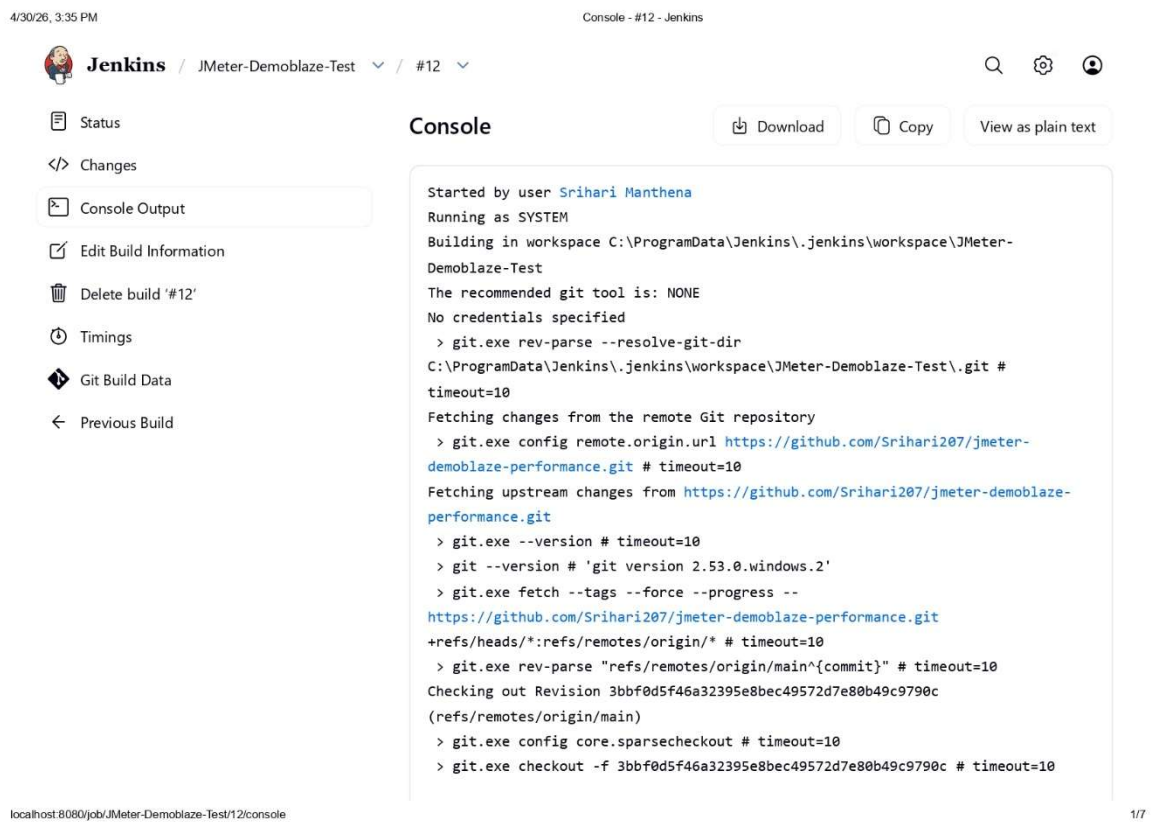
This image show the Repository in GitHub ready to pull



This image show the Repository Pulled from the GitHub



This images the Build Steps in Jenkins



4/30/26, 3:35 PM

Console - #12 - Jenkins

```
Commit message: "Update test flow format in README"
> git.exe rev-list --no-walk 3bbf0d5f46a32395e8bec49572d7e80b49c9790c #
timeout=10
[JMeter-Demoblaze-Test] $ cmd /c call
C:\WINDOWS\TEMP\jenkins2496420492443521994.bat

C:\ProgramData\Jenkins\.jenkins\workspace\JMeter-Demoblaze-Test>set
JAVA_HOME=C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot

C:\ProgramData\Jenkins\.jenkins\workspace\JMeter-Demoblaze-Test>set
PATH=C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin;C:\Windows\System32;C:\Program Files\Eclipse Adoptium\jdk-25.0.2.10-hotspot\bin;C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin;C:\Program Files\Java\jdk-17;C:\Program Files\Java\jdk-17\bin;C:\Program Files\Git\cmd;C:\Program Files\Java\jdk-17\bin;C:\Program Files\Java\jdk-17\bin;C:\WINDOWS\system32\config\systemprofile\AppData\Local\Microsoft\WindowsApps

C:\ProgramData\Jenkins\.jenkins\workspace\JMeter-Demoblaze-Test>cd C:\apache-jmeter-5.6.3\bin

C:\apache-jmeter-5.6.3\bin>if exist
C:\ProgramData\Jenkins\.jenkins\workspace\JMeter-Demoblaze-Test\results.jtl del
C:\ProgramData\Jenkins\.jenkins\workspace\JMeter-Demoblaze-Test\results.jtl

C:\apache-jmeter-5.6.3\bin>if exist
C:\ProgramData\Jenkins\.jenkins\workspace\JMeter-Demoblaze-Test\report rmdir /s
/q C:\ProgramData\Jenkins\.jenkins\workspace\JMeter-Demoblaze-Test\report
```

localhost:8080/job/JMeter-Demoblaze-Test/12/console

2/7

4/30/26, 3:35 PM

Console - #12 - Jenkins

```
C:\apache-jmeter-5.6.3\bin>jmeter -n -t
"C:\ProgramData\Jenkins\.jenkins\workspace\JMeter-Demoblaze-Test\demoblaze.jmx"
-l "C:\ProgramData\Jenkins\.jenkins\workspace\JMeter-Demoblaze-Test\results.jtl"
-e -o "C:\ProgramData\Jenkins\.jenkins\workspace\JMeter-Demoblaze-Test\report"
WARN StatusConsoleListener The use of package scanning to locate plugins is
deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is
deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is
deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is
deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is
deprecated and will be removed in a future release
Creating summariser <summary>
Created the tree successfully using
C:\ProgramData\Jenkins\.jenkins\workspace\JMeter-Demoblaze-Test\demoblaze.jmx
Starting standalone test @ 2026 Apr 30 14:18:43 IST (1777538923354)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port
4445
summary +      54 in 00:00:16 =    3.4/s Avg:   374 Min:   110 Max:  1749 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary +     124 in 00:00:30 =    4.1/s Avg:   297 Min:    85 Max:  1036 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary =     178 in 00:00:46 =    3.9/s Avg:   320 Min:    85 Max:  1749 Err:
0 (0.00%)
summary +     131 in 00:00:30 =    4.3/s Avg:   334 Min:    76 Max:  1066 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary =     309 in 00:01:16 =    4.1/s Avg:   326 Min:    76 Max:  1749 Err:
0 (0.00%)
summary +      99 in 00:00:30 =    3.3/s Avg:  1140 Min:    74 Max: 11919 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
```

localhost:8080/job/JMeter-Demoblaze-Test/12/console

3/7

4/30/26, 3:35 PM

Console - #12 - Jenkins

```
summary = 408 in 00:01:46 = 3.8/s Avg: 524 Min: 74 Max: 11919 Err:
0 (0.00%)
summary + 125 in 00:00:30 = 4.2/s Avg: 357 Min: 87 Max: 933 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 533 in 00:02:16 = 3.9/s Avg: 485 Min: 74 Max: 11919 Err:
0 (0.00%)
summary + 127 in 00:00:30 = 4.2/s Avg: 359 Min: 73 Max: 2191 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 660 in 00:02:46 = 4.0/s Avg: 460 Min: 73 Max: 11919 Err:
0 (0.00%)
summary + 129 in 00:00:30 = 4.3/s Avg: 350 Min: 94 Max: 912 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 789 in 00:03:16 = 4.0/s Avg: 442 Min: 73 Max: 11919 Err:
0 (0.00%)
summary + 132 in 00:00:30 = 4.4/s Avg: 326 Min: 70 Max: 722 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 921 in 00:03:46 = 4.1/s Avg: 426 Min: 70 Max: 11919 Err:
0 (0.00%)
summary + 128 in 00:00:30 = 4.3/s Avg: 340 Min: 80 Max: 1422 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 1049 in 00:04:16 = 4.1/s Avg: 415 Min: 70 Max: 11919 Err:
0 (0.00%)
summary + 128 in 00:00:30 = 4.3/s Avg: 332 Min: 74 Max: 686 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 1177 in 00:04:46 = 4.1/s Avg: 406 Min: 70 Max: 11919 Err:
0 (0.00%)
summary + 126 in 00:00:30 = 4.2/s Avg: 332 Min: 80 Max: 919 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 1303 in 00:05:16 = 4.1/s Avg: 399 Min: 70 Max: 11919 Err:
0 (0.00%)
```

localhost:8080/job/JMeter-Demoblaze-Test/12/console

4/7

4/30/26, 3:35 PM

Console - #12 - Jenkins

```
summary + 133 in 00:00:30 = 4.4/s Avg: 340 Min: 89 Max: 658 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 1436 in 00:05:46 = 4.1/s Avg: 394 Min: 70 Max: 11919 Err:
0 (0.00%)
summary + 123 in 00:00:30 = 4.1/s Avg: 339 Min: 85 Max: 655 Err:
3 (2.44%) Active: 10 Started: 10 Finished: 0
summary = 1559 in 00:06:16 = 4.1/s Avg: 389 Min: 70 Max: 11919 Err:
3 (0.19%)
summary + 132 in 00:00:31 = 4.3/s Avg: 334 Min: 92 Max: 651 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 1691 in 00:06:47 = 4.2/s Avg: 385 Min: 70 Max: 11919 Err:
3 (0.18%)
summary + 122 in 00:00:30 = 4.1/s Avg: 311 Min: 0 Max: 902 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 1813 in 00:07:16 = 4.2/s Avg: 380 Min: 0 Max: 11919 Err:
3 (0.17%)
summary + 128 in 00:00:30 = 4.3/s Avg: 303 Min: 90 Max: 752 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 1941 in 00:07:46 = 4.2/s Avg: 375 Min: 0 Max: 11919 Err:
3 (0.15%)
summary + 124 in 00:00:30 = 4.1/s Avg: 320 Min: 81 Max: 896 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 2065 in 00:08:16 = 4.2/s Avg: 371 Min: 0 Max: 11919 Err:
3 (0.15%)
summary + 125 in 00:00:31 = 4.1/s Avg: 367 Min: 88 Max: 1217 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 2190 in 00:08:47 = 4.2/s Avg: 371 Min: 0 Max: 11919 Err:
3 (0.14%)
summary + 136 in 00:00:30 = 4.6/s Avg: 311 Min: 81 Max: 968 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
```

localhost:8080/job/JMeter-Demoblaze-Test/12/console

5/7

4/30/26, 3:35 PM

Console - #12 - Jenkins

```
summary = 2326 in 00:09:16 = 4.2/s Avg: 368 Min: 0 Max: 11919 Err:
3 (0.13%)
summary + 127 in 00:00:30 = 4.3/s Avg: 376 Min: 88 Max: 1648 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 2453 in 00:09:46 = 4.2/s Avg: 368 Min: 0 Max: 11919 Err:
3 (0.12%)
summary + 127 in 00:00:30 = 4.3/s Avg: 335 Min: 72 Max: 911 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 2580 in 00:10:16 = 4.2/s Avg: 367 Min: 0 Max: 11919 Err:
3 (0.12%)
summary + 120 in 00:00:30 = 4.0/s Avg: 419 Min: 82 Max: 2071 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 2700 in 00:10:46 = 4.2/s Avg: 369 Min: 0 Max: 11919 Err:
3 (0.11%)
summary + 129 in 00:00:30 = 4.3/s Avg: 334 Min: 83 Max: 770 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 2829 in 00:11:16 = 4.2/s Avg: 367 Min: 0 Max: 11919 Err:
3 (0.11%)
summary + 121 in 00:00:30 = 4.0/s Avg: 439 Min: 79 Max: 1877 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 2950 in 00:11:47 = 4.2/s Avg: 370 Min: 0 Max: 11919 Err:
3 (0.10%)
summary + 129 in 00:00:29 = 4.4/s Avg: 330 Min: 90 Max: 662 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 3079 in 00:12:16 = 4.2/s Avg: 369 Min: 0 Max: 11919 Err:
3 (0.10%)
summary + 124 in 00:00:30 = 4.1/s Avg: 372 Min: 77 Max: 1452 Err:
0 (0.00%) Active: 10 Started: 10 Finished: 0
summary = 3203 in 00:12:46 = 4.2/s Avg: 369 Min: 0 Max: 11919 Err:
3 (0.09%)
```

localhost:8080/job/JMeter-Demoblaze-Test/12/console

6/7

4/30/26, 3:35 PM

Console - #12 - Jenkins

```
summary + 132 in 00:00:30 = 4.4/s Avg: 316 Min: 77 Max: 558 Err:
2 (1.52%) Active: 10 Started: 10 Finished: 0
summary = 3335 in 00:13:16 = 4.2/s Avg: 367 Min: 0 Max: 11919 Err:
5 (0.15%)
summary + 131 in 00:00:30 = 4.4/s Avg: 355 Min: 1 Max: 1818 Err:
0 (0.00%) Active: 9 Started: 10 Finished: 1
summary = 3466 in 00:13:46 = 4.2/s Avg: 366 Min: 0 Max: 11919 Err:
5 (0.14%)
summary + 34 in 00:00:21 = 1.7/s Avg: 495 Min: 0 Max: 2517 Err:
0 (0.00%) Active: 0 Started: 10 Finished: 10
summary = 3500 in 00:14:07 = 4.1/s Avg: 367 Min: 0 Max: 11919 Err:
5 (0.14%)
Tidying up ... @ 2026 Apr 30 14:32:50 IST (1777539770639)
... end of run
Finished: SUCCESS
```

Jenkins 2.541.3

localhost:8080/job/JMeter-Demoblaze-Test/12/console

7/7

11.RESULTS & ANALYSIS

11.1 Execution Summary

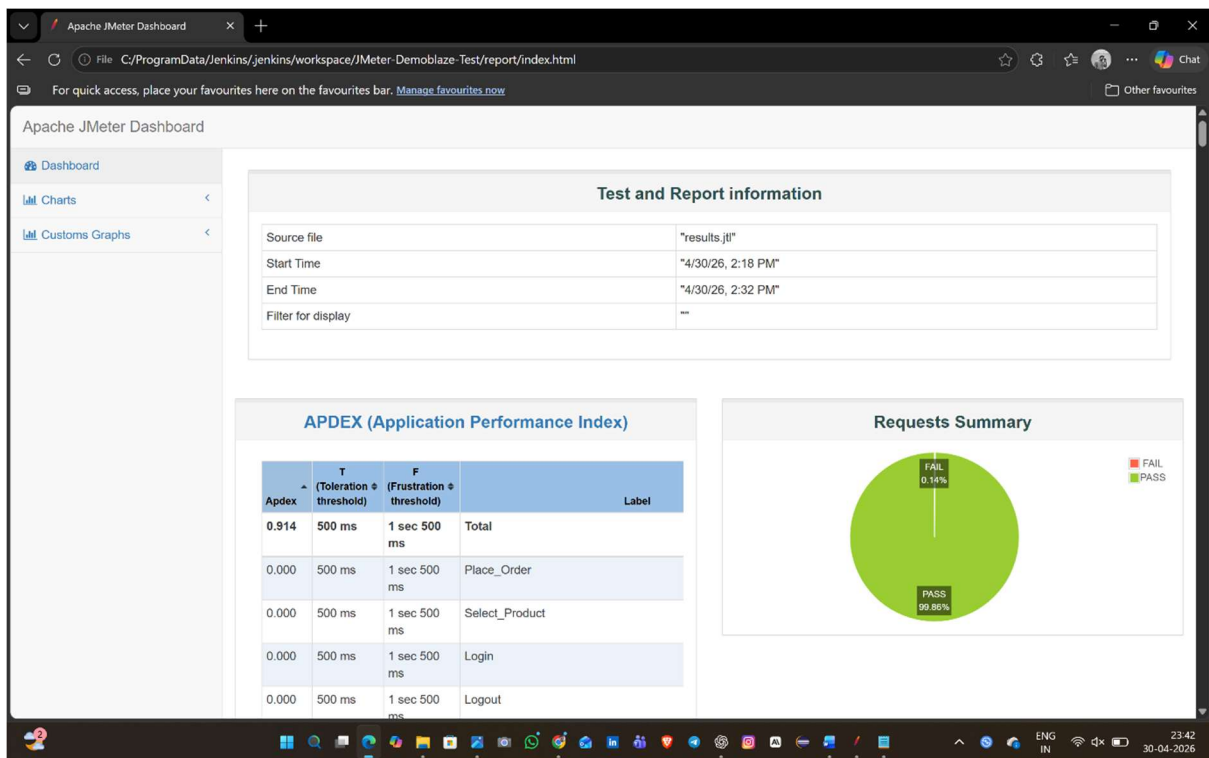
The performance test was successfully executed using Jenkins CI/CD pipeline. JMeter ran in non-GUI mode and generated results along with HTML reports.

11.2 Test Statistics

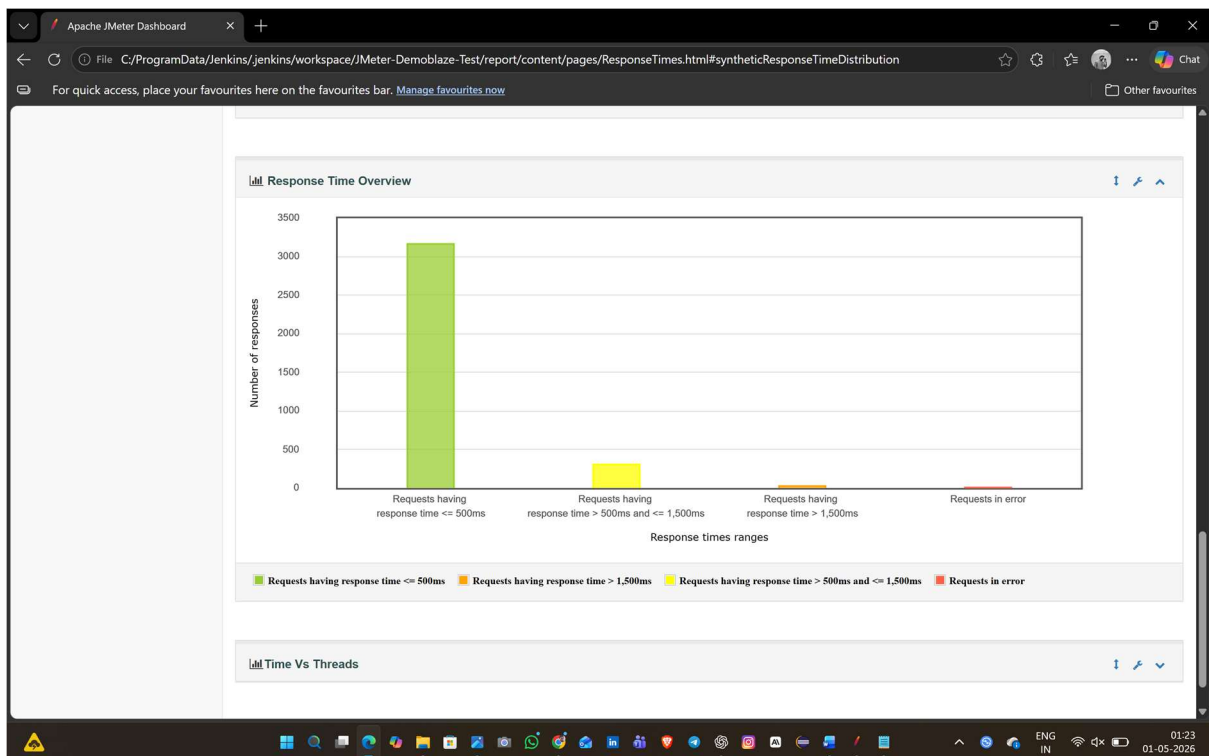
Metric	Value
Total Samples	3500
Test Duration	~14 minutes
Average Response Time	~367 ms
Minimum Response Time	0 ms
Maximum Response Time	~11919 ms
Throughput	~4.1 – 4.2 requests/sec
Error Count	5
Error Percentage	~0.14%

11.3 Observations

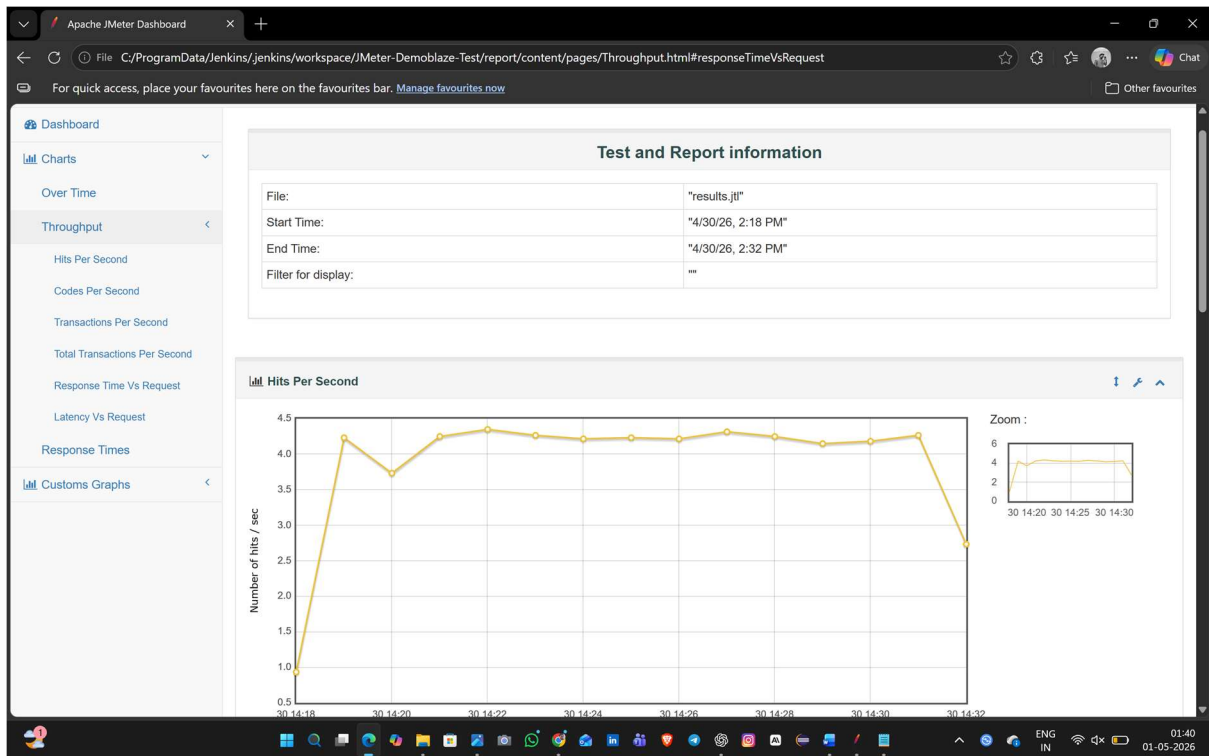
- The application handled **10 concurrent users** efficiently
- Most response times were **under 400 ms**, indicating good performance
- A few spikes were observed with higher response time (~11 sec)
- Error rate was very low (<1%), which indicates system stability
- Throughput remained consistent throughout execution
- The response time spikes (~11 sec) indicate potential backend delays under peak load conditions.



1.JMeter Dashboard Overview



2.Response Time Graph



3.Throughput Graph

12.CHALLENGES FACED:

- Handling dynamic tokens during correlation
- Debugging failed requests due to incorrect extraction
- Configuring Jenkins pipeline for automated execution
- Managing realistic user simulation using parameterization
-

13.CONCLUSION:

The system performed well under the given load conditions with stable response time and minimal errors. The results indicate that the application can handle moderate user load efficiently.

14.FUTURE ENHANCEMENTS:

- Increase load to simulate high traffic scenarios (100+ users)
- Integrate **Grafana + InfluxDB** for real-time monitoring
- Implement distributed load testing
- Add performance trend analysis across builds